SEGP - Team Agreements

cmk123, akc123, kmt123, jhc23, rg1523, rlc23

January 13, 2026

1 What is the overall goal of your project?

Our goal is to develop an adaptive Speed Reading web platform that enhances reading speed and comprehension using eye-tracking technology. The system will integrate WebGazer.js for real-time webcam-based eye tracking to monitor user engagement and pace. We will implement three reading modes: a fixed WPM (Words Per Minute) mode for beginners, an adaptive mode that adjusts speed based on eye-tracking data, and a summarised mode using Generative AI for non-fiction content. The platform will include comprehension quizzes, a repository of public domain texts with difficulty ratings, user dashboards for progress tracking, and an admin interface for content management.

2 Team composition: why did you choose the team members you did? Do you have a good balance of skills? Have you allocated any specific roles to particular members?

Our team has a diverse set of skills relevant to both the technical and organisational demands of the project. Through previous group projects and internship experiences, we collectively have experience in front-end and back-end development, database design, and the technologies in our stack.

Initial responsibilities have been assigned based on individual strengths, with flexibility to adapt as the project progresses. For example, team members with front-end experience will lead the React UI development, while those with database experience will handle Supabase integration.

3 How many hours per week have you each committed to work on the project?

Since this module is equivalent to two standard modules (10 ECTS), each team member is expected to contribute approximately 250 hours over the course of the project, equating to roughly 25 hours per week or 5 hours per working day. With six team members, it is impractical for everyone to work at mutually exclusive times, making the ability to collaborate and work in parallel essential.

4 Are you adopting a specific software engineering method or process in your team (e.g. Scrum, XP, Kanban)? What does this mean in practical terms for your team and the way that you will work?

We will adopt **Scrum** as our software engineering methodology. Scrum provides an iterative, incremental framework well-suited for our project's evolving requirements, particularly given the experimental nature of integrating eye-tracking with adaptive reading systems.

In practical terms, we will work in two-week sprints. Each sprint begins with a planning session where we select items from our product backlog and commit to deliverables. Daily stand-ups (or asynchronous updates via our communication channel) will keep the team aligned on progress and blockers. At the end of each sprint, we will conduct a review to demonstrate working features and a retrospective to improve our process.

We will maintain a product backlog prioritised by feature importance and technical dependencies. The WebGazer integration and core reading modes will be prioritised early, followed by the comprehension assessment system and admin interface. This approach allows us to deliver a working vertical slice of the application early and iterate based on feedback.

5 What (if any) technology will you use for a) source control, b) tracking the progress of work in your project, c) communication amongst team members, d) storing and sharing documents

- **Source Control:** GitLab - for version control, code hosting, merge requests, and code reviews. We will use feature branches and require reviews before merging to main.
- **Progress Tracking:** GitLab Issues - for managing our Scrum backlog, sprint planning, and tracking task status. Issues will be labelled and moved through stages (To Do, In Progress, Review, Done).
- **Communication:** Discord and WhatsApp - Discord for screen sharing, voice calls, and organised discussions with dedicated channels; WhatsApp for quick updates and urgent coordination.
- **Document Storage:** One Drive and Overleaf - for storing and sharing project documents, meeting notes, design assets and reports. Documents may also be shared via Discord or WhatsApp when needed.

6 Have you set up a schedule for regular team meetings? When will they be?

We have used `when2meet` to check when most of our team members are free throughout the week, and decided that we are all free during the scheduled "Y3 Group Project (meeting slot)", as well as on other days such as Wednesday afternoon. This will be supplemented by our daily stand-up meetings to discuss progress throughout each sprint. We will constantly be in touch with our customer to set up checkpoints during each sprint to ensure our work is aligned with our customer's needs.

7 Have you set any team rules?

In terms of meetings and attendance, all team members are expected to attend in-person meetings and working sessions with our customer, as these are critical for gathering requirements, receiving feedback, and aligning expectations. Attendance at daily stand-up meetings can be missed provided there is a valid reason since all members in the group have different schedules. However, the member should communicate their progress and blockers to the rest of the team in advance or via an agreed asynchronous channel to ensure progress.

Regarding code quality and development practices, we have established a number of rules to maintain consistency and readability across the codebase. For TypeScript, we use Prettier to enforce a consistent coding style, while for Python code we use Black for automatic formatting. All code should be clearly structured, appropriately modularised, and include meaningful comments where the logic is non-trivial. This helps ensure the code remains understandable and maintainable by all team members.

We will follow a pull request-based workflow for all non-trivial changes. Code must be submitted via a pull request and thoroughly reviewed by at least one other team member before being merged. Reviewers are expected to carefully read through the code, check for correctness, clarity, and potential issues, and suggest improvements where appropriate. Automated tests are encouraged where feasible, particularly for core functionality, and all existing tests must pass before a pull request can be approved.

8 All teams will complete a simple peer-review exercise at the end of each iteration. You'll need to say whether each team member's contribution, matches, exceeds, or falls short of the team's expectations. What will be the criteria by which you judge each other's contribution to the project?

Using the Feedback Fruits system, each team member will rate the contributions of their peers after each iteration on a scale of 0 (no contribution) to 3 (met expectations). We will evaluate each other based on: code contributions, planning and organisation efforts (e.g. Scrum Master duties), code reviews conducted, and documentation or report writing completed.

9 Give a brief overview of your initial project plan.

Our project will be divided into four two-week sprints, following our implementation phases:

Sprint 1: Foundation, Infrastructure, and Core Reading

- Set up monorepo structure (React + Vite + TypeScript + Tailwind CSS + React Router frontend, Supabase backend)
- Configure CI/CD pipeline with GitLab CI for automated testing and deployment
- Provision infrastructure: Supabase (PostgreSQL, auth, API), Vercel (frontend)
- Implement user authentication using Supabase Auth and define database schema
- Develop text display component with word highlighting
- Implement Mode 1 (Standard Speed-Reading) with fixed WPM and manual controls
- Seed database with initial public domain texts

Sprint 2: WebGazer Integration and Adaptive Mode

- Integrate WebGazer.js for webcam-based eye tracking
- Implement 9-point calibration flow with validation (fallback if <50% accuracy)
- Develop gaze region detection (top/middle/bottom thirds) with temporal smoothing
- Build adaptive WPM controller based on gaze data
- Implement Mode 2 (Adaptive Speed-Reading) with scroll-based fallback
- Create user control panel for manual adjustments

Sprint 3: AI Features and Comprehension System

- Set up cloud AI API integration (Groq or Together AI) for open-source models
- Integrate AI API for quiz generation and text summarisation
- Implement Mode 3 (Summarised Adaptive Reading) for non-fiction texts
- Develop quiz generation and admin approval workflow
- Create quiz UI with scoring system

Sprint 4: Gamification, Polish, and Deployment

- Implement XP/levelling system and achievements
- Develop user dashboard with progress charts and reading history
- Create admin interface for text and quiz management
- Comprehensive cross-browser testing (Chrome, Firefox, Edge, Safari)
- Final documentation and presentation preparation

10 What are the main risks to the success of this project?

Risk	**Severity**	**Mitigation Strategy**
WebGazer accuracy issues (lighting, webcam quality, user positioning)	High	Use coarse region detection instead of word-level tracking; implement 9-point calibration with validation; force fallback mode if accuracy <50%; use 2-3 second temporal smoothing
Browser compatibility (getUserMedia, TensorFlow.js differences)	Medium	Focus initial development on Chrome; conduct cross-browser testing each sprint; maintain compatibility matrix for Chrome, Firefox, Edge, Safari
AI API availability (rate limits, network issues)	Medium	Pre-generate summaries and quizzes for MVP; cache AI-generated content in databases; store approved quizzes rather than generating on-demand
Scope creep (features expanding beyond 4 sprints)	High	Strictly prioritise MVP features; use MoSCoW method; defer nice-to-have features; regular backlog grooming in sprint planning
Team member availability (schedules, illness, commitments)	Medium	Cross-train on critical components; document code thoroughly; clear task ownership via GitLab Issues; buffer time in sprint planning
Performance issues (real-time eye tracking causing lag)	Medium	Profile performance early; use web workers for computation; set minimum hardware requirements; implement scroll-based fallback
User privacy concerns (webcam access deterring users)	Low	All gaze data processed client-side only; no video sent to server; clear privacy documentation; only aggregated metrics stored

11 Signatures

Team Member:

Caleb Kan

Team Member:

Alexander Chow

Team Member:

Kee Meng Tan

Team Member:

Jeffrey Cheung

Team Member:

Ryan Ghoussainy

Team Member:

Royce Chan

Project Supervisor:

Konstantinos Gkoutzis